" राम - राम "

Link : De Shaw OA / CTC SOL - Desi QnA

Understanding : We are given an array of size n and max score m , we have to find the longest subsequence [a[i1], a[i2], ... , a[ik]] such that score of this subsequence is <= m

Score = i1*k + a[i1] + i2*k + a[i2] + ... + ik*k + a[ik] <= m

Ex : [4,3,2,1], m = 33

O/p = 3 → [1,2,4] idx array

$1 <= n <= 2*10^5$

$1 <= a[i] <= 10^4$

$1 <= max\ score <= 10^{15}$

We know that score of the subsequence should be <= m

So organically we should think of starting with the length 1

Let's start by taking subsequence of length 1

All subsequences of length 1 → 1*1 + a[1] , 2*1 + a[2], 3*1 + a[3], ..., n*1 + a[n]

So we will take smallest i*1 + a[i] such that our score is <= m

Now we will take length 2 of subsequence

i.e we need 2 elements such that their score is <= m

Score = (i * 2 + a[i]) + (j * 2 + a[j])

So we can see that if we are able to find the two smallest values whose sum <= m then our length 2 subsequence is valid

Similarly we will check for length 3 , 4, ... , till we our getting score of that subsequence <= m if we found a length which is not giving score <= m then we will stop

Brute Force :

Start with len = 1 ..... Len = length of array i.e n

Check for each len if score <= m increase the length else stop and return current len

Code : https://onecompiler.com/java/456u7b8nv

T.C = O(N*N*log(N))

S.C = O(N)

Optimized :

Using Binary Search on subsequence length

Previous code will remain the same the only change will be in loop from which we are calling the helper function

T.C = O(log(N) * N* log(N)) = O(N* log(N)* log(N))

Code : https://onecompiler.com/java/456ujhkgf